UNIVERSITY OF MINDANAO TAGUM BRANCH
BACHELOR OF SCIENCE IN COMPUTER SCIENCE
Department of Computing Education
Visayan Village, Tagum City, Philippines

Course Requirements
in
Programming Languages (CS 15/L - 1831)

EVENTRA: Attendance Management System for School Events

Submitted to:

XIAN RHEL S. CADIOGAN

Submitted by:

BUENA, GABRIEL JEE M.
MUNDIZ, KENN JB A.
SAN JOSE, ALEXIE BRYCE C.

October 26, 2026

APP DESCRIPTION

EVENTRA: Attendance Management System for School Events is a mobile application designed to simplify and improve the process of managing school events and recording student attendance through QR code system. It provides separate accounts for school officers and students (attendees), each with specific features and functionalities:

- The **Officer Account** allows authorized officers to create and manage school events, record student attendance by scanning QR codes, view event details and attendees list, edit attendee records, and archive or delete events. These features help officers manage school activities and monitor student participation more efficiently.
- The **Student Account** allows students to create their own accounts, generate personal QR codes for attendance verification, and view available school activities while making the attendance process faster and more organized.

The application aims to reduce the time and effort required for manual attendance recording, minimize errors in attendance tracking, and provide a more efficient and organized system for managing school events.

KEY FEATURES

Google sign-in and account setup

- Students and officers authenticate with verified Google accounts. The app matches officers against a seeded roster and assigns a student ID.

Role-based access

- Students can view events and their attendance. Officers can manage events and record attendance; only officers can access event archives.

Event management

- Officers can create and edit events with a name, dates, location, and description. They can also archive events.

Event discovery and search

- Users can browse active events, search by keyword, and view event details, including dates, location, and description.

QR attendance check-in

- Students can display a QR code containing their student ID. Officers can scan it or enter the ID manually to record attendance.

Attendance records and corrections

- Officers can review an event's attendee list, search for attendees, correct check-in time or scanner name, and remove attendance records.

Attendance history

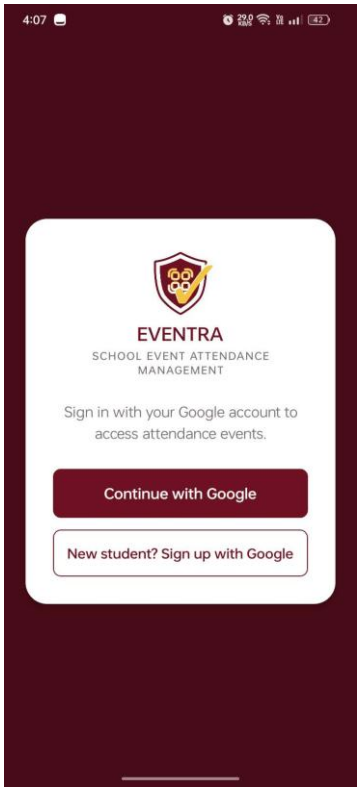
- Students can review their recorded attendance, including the event details, check-in time, and officer who recorded it. History includes archived events.

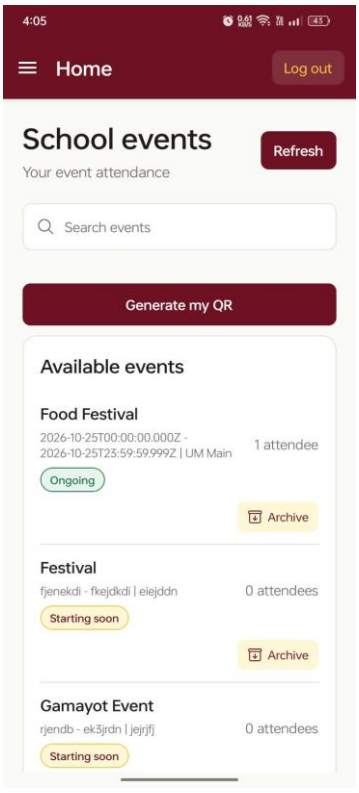
Event archives

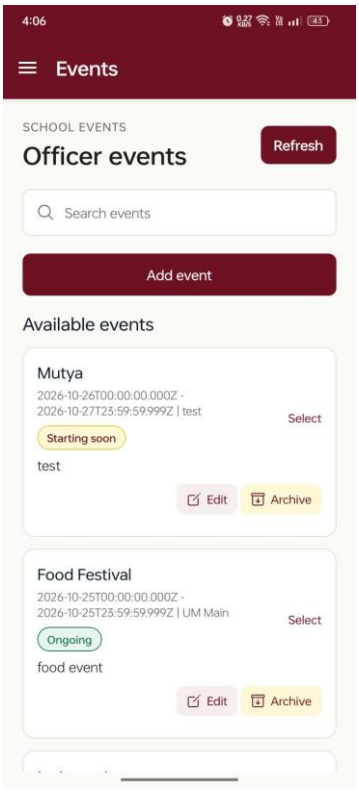
- Officers can review archived events and see their archive details and attendee lists.

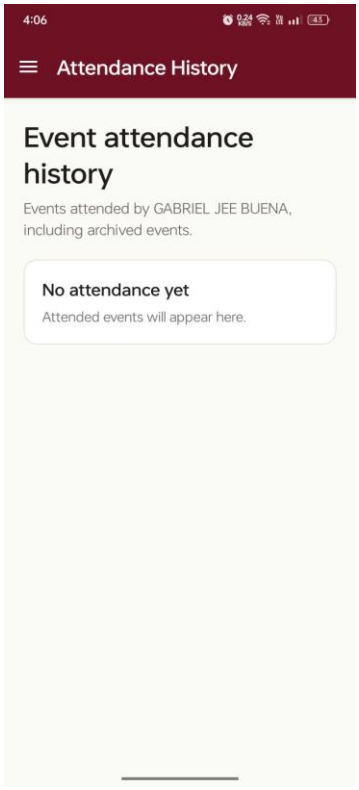
SCREENSHOT OF MAJOR SCREENS

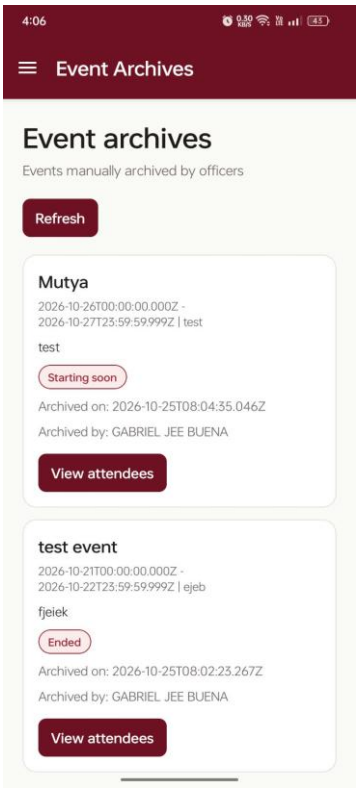
Officer Side

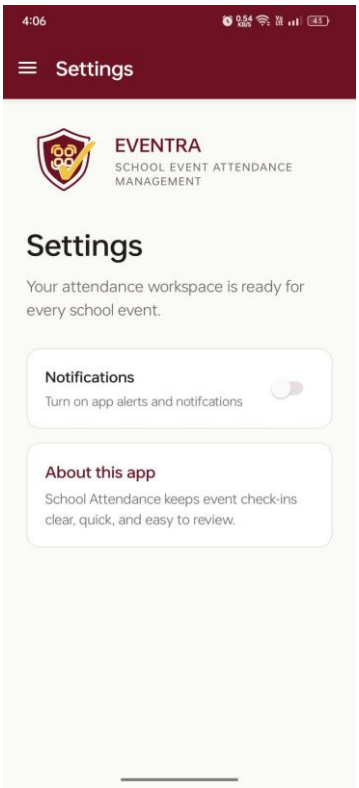




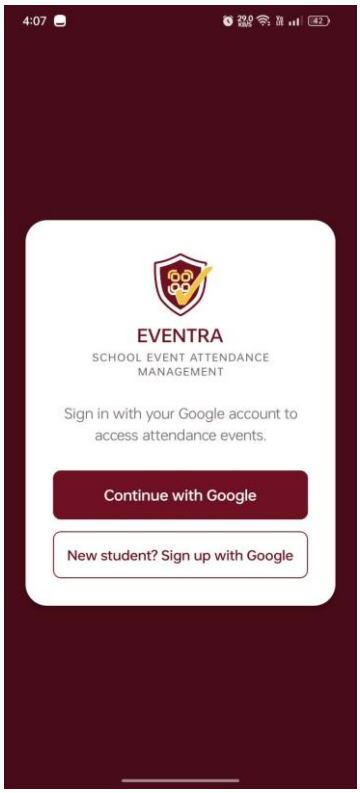


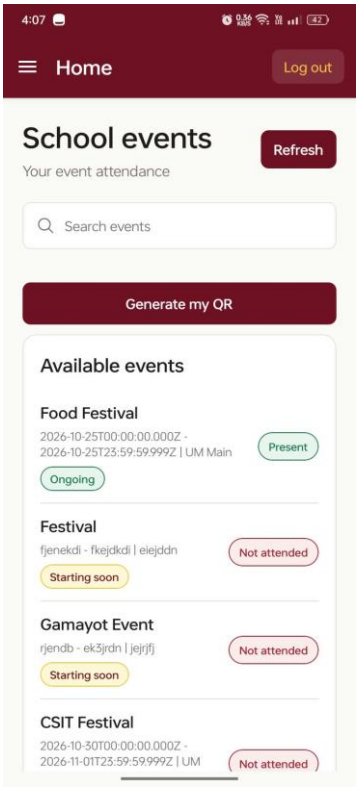


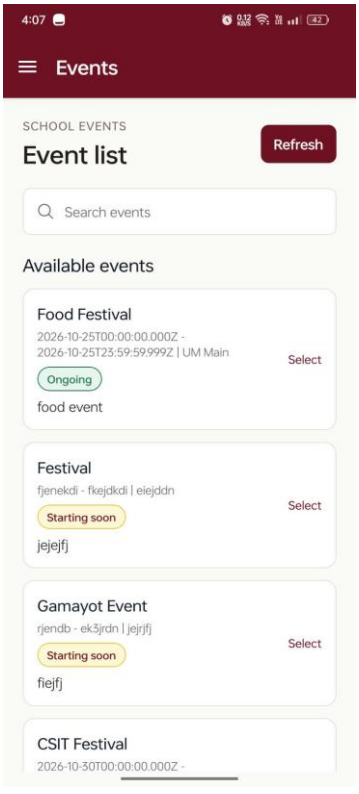


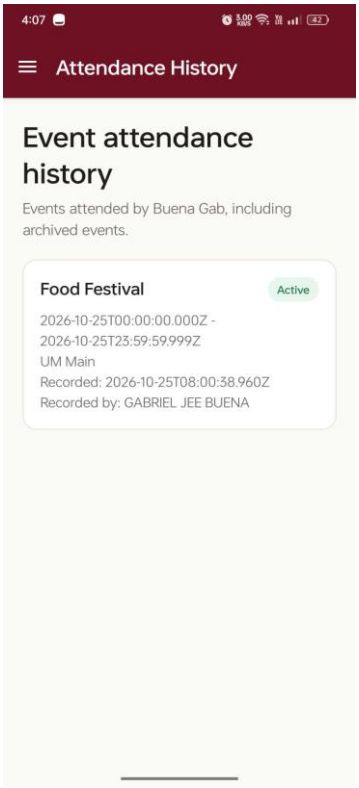


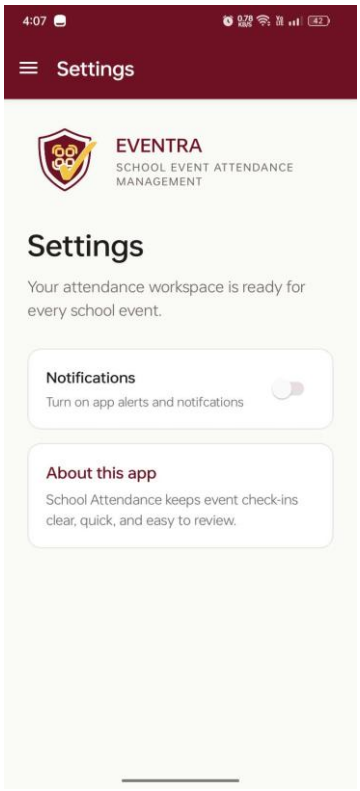
Student Side











SYSTEM ARCHITECTURE

The Attendance Management System for School Events follows a layered, full-stack architecture with a clear separation between the client and server. The project is organized into three main parts: shared assets, a backend, and a frontend.

The **backend** follows a controller-service-repository pattern. Incoming requests pass through the *routes*, which hand them to the *controllers*. The controllers delegate business logic to the *services*, which in turn use the *repositories* to read from and write to the *data* layer. Shared *types* keep the data structures consistent across all layers, and server.ts serves as the entry point that brings everything together. This structure covers authentication, event management, attendance recording, and health checks.

The **frontend** is a TypeScript React Native (Expo) application. It is divided into *screens* (Home, Login, Signup, Event, Attendance History, Event Archives, and Settings), reusable *components* such as the student QR modal, an *api* layer for communicating with the backend, and *auth* handling for Google sign-in. *Navigation* is managed through a drawer menu, while dedicated *styles* and a central theme keep the interface visually consistent.

The **assets** folder stores the app logos, icons, and splash screen, and the root configuration files (package.json, tsconfig.json, app.json, and eas.json) handle dependencies, TypeScript settings, and build configuration.

```
Attendance-Management-System-for-School-Events/
├── assets/
│   ├── AppLogo2.png
│   ├── AppLogo3.png
│   ├── AppLogoQR_draft.png
│   ├── adaptive-icon.png
│   ├── favicon.png
│   ├── icon.png
│   └── splash-icon.png
├── backend/
│   ├── controllers/
│   │   ├── attendances.controller.ts
│   │   ├── auth.controller.ts
│   │   ├── events.controller.ts
│   │   ├── health.controller.ts
│   │   └── http.ts
│   ├── data/
│   │   ├── database.ts
│   │   ├── date-time.ts
│   │   └── identity.ts
│   ├── repositories/
│   │   ├── attendances.repository.ts
│   │   ├── auth.repository.ts
│   │   └── events.repository.ts
│   ├── routes/
│   │   ├── attendances.routes.ts
│   │   ├── auth.routes.ts
│   │   ├── events.routes.ts
│   │   └── health.routes.ts
│   ├── services/
│   │   ├── attendances.service.ts
│   │   ├── auth.service.ts
│   │   └── events.service.ts
│   ├── types/
│   │   ├── attendance.ts
│   │   ├── auth.ts
│   │   ├── event.ts
│   │   └── service-error.ts
│   └── server.ts
├── frontend/
│   ├── api/
│   │   ├── auth.ts
│   │   ├── config.ts
│   │   └── events.ts
│   ├── auth/
│   │   └── useGoogleSignIn.ts
│   ├── components/
│   │   ├── BrandMark.tsx
│   │   └── StudentQrModal.tsx
│   ├── mock-data/
│   │   └── mockAttendance.ts
│   ├── navigation/
│   │   └── DrawerNavigation.tsx
│   ├── screens/
│   │   ├── AttendanceHistoryScreen.tsx
│   │   ├── EventArchivesScreen.tsx
│   │   ├── EventScreen.tsx
│   │   ├── HomeScreen.tsx
│   │   ├── LoginScreen.tsx
│   │   ├── SettingsScreen.tsx
│   │   └── SignupScreen.tsx
│   ├── styles/
│   │   ├── DrawerStyles.style.ts
│   │   ├── EventScreen.style.ts
│   │   ├── GlobalStyles.style.ts
│   │   ├── HomeScreen.style.ts
│   │   ├── LoginScreen.style.ts
│   │   ├── SettingsScreen.style.ts
│   │   └── theme.ts
│   └── types/
│       └── event.ts
├── App.tsx
├── index.js
├── package.json
├── tsconfig.json
├── app.json
├── eas.json
├── components/
│   └── AssetExample.js
├── package-lock.json
└── README.md
```

NAVIGATION FLOW

Navigation Structure

```
None
Login
├── Signup
└── Main
    └── Drawer
        ├── Home
        ├── Events
        ├── Attendance History
        ├── Event Archives  (Officer only)
        └── Settings
```

Authentication Flow

1. The app opens on the Login screen.
2. Existing users select Continue with Google.
3. New students can select Sign up with Google to open Signup.
4. After successful Google authentication, the account information and role are passed to Main.
5. Authentication errors are displayed on the current screen.

Users can return from Signup to Login using Back to login.

Main Screens

Screen	Description
Home	Displays active events. Users can search, refresh, and view event details. Students can generate a QR code; officers can manage attendance and archive events.
Events	Students can view events. Officers can create, edit, archive, and manage attendance using manual entry or QR scanning.
Attendance History	Shows the signed-in student's attendance records, including archived events.
Event Archives	Available only to officers. Displays archived events and their attendees.
Settings	Contains the notification toggle and app information.

Role-Based Access

Feature	Student	Officer
View Events	Yes	Yes
Generate personal QR	Yes	If a student ID is available
Create/edit events	No	Yes

Record Attendance	No	Yes
Edit/delete attendance	No	Yes
Archive events	No	Yes
View event archives	No	Yes

Local Modal Flows

Some interactions stay within the current screen instead of creating a new route. These include:

- Student QR code
- Attendee lists
- QR scanner
- Date picker
- Delete and archive confirmations
- Archived-event attendee details

Logout Flow

From the Home screen, selecting Log out returns the user to Login.

DATABASE STRUCTURE

The application uses a local SQLite database. The database file is DummySample.db by default, or the path provided through ATTENDANCE_DB_PATH.

```
None
students  1 ———< attendances >——— 1  events

officers  (staff roster used for officer role matching)
```

Tables

- **students –** Stores student accounts created or updated through Google authentication.

Column	Description
student_id	Primary key and student identifier
full_name	Complete student name
given_name	First or given name

last_name	Family name
email	Google account email; unique
google_subject	Google account subject ID; unique
created_at	Account creation timestamp

- **officers -** Stores the officer roster used to identify users with officer access.

Column	Description
id	Primary key
given_name	Officer's given name
last_name	Officer's family name
matching_name	Normalized name used for matching; unique
student_id	Optional linked student ID; unique
created_at	Record creation timestamp
updated_at	Last update timestamp

- **events –** Stores school events.

Column	Description
id	Primary key
name	Event name
description	Event details; optional
start_date / end_date	Event date range
location	Event location
created_at	Creation timestamp
status	starting_soon, ongoing, or ended
archived_at	Archive timestamp; null while active
archived_by	Officer who archived the event

- **attendances -** Stores the officer roster used to identify users with officer access.

Column	Description
id	Primary key
event_id	References **events.id**

student_id	References **students.student_id**
scanned_by	Person who recorded the attendance
status	Defaults to **present**
scanned_at	Attendance timestamp

An event and student pair must be unique, so a student cannot be recorded twice for the same event.

Relationships

- One student can have many attendance records.
- One event can have many attendance records.
- Each attendance record belongs to one student and one event.
- Attendance uses foreign keys to connect students and events.
- Foreign-key enforcement is enabled with SQLite.

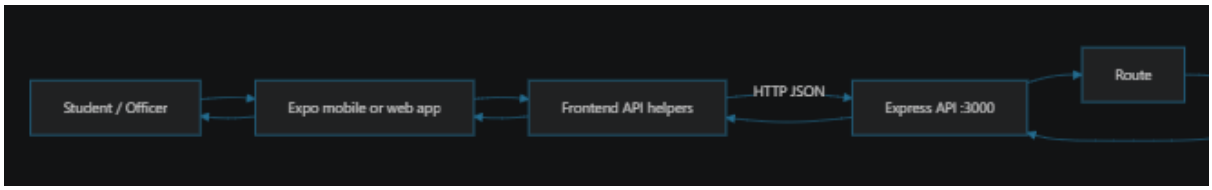
Main Database Operations

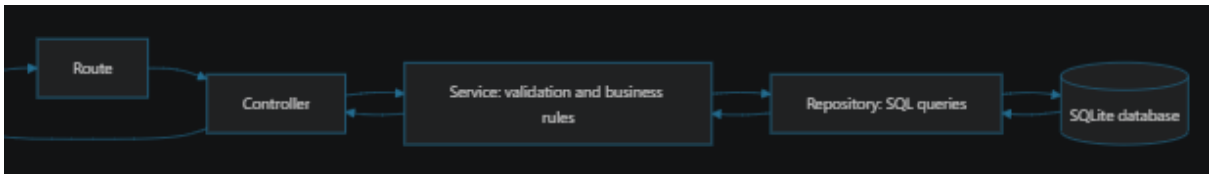
- Google sign-in creates or updates a record in **students.**
- Officers can create, update, list, and archive records in **events.**
- Attendance can be recorded, viewed by event or student, updated, and deleted.
- Event status is updated based on the current date when events are listed.
- Archived events remain in the database but are separated from active event lists.

DATABASE / API FLOW

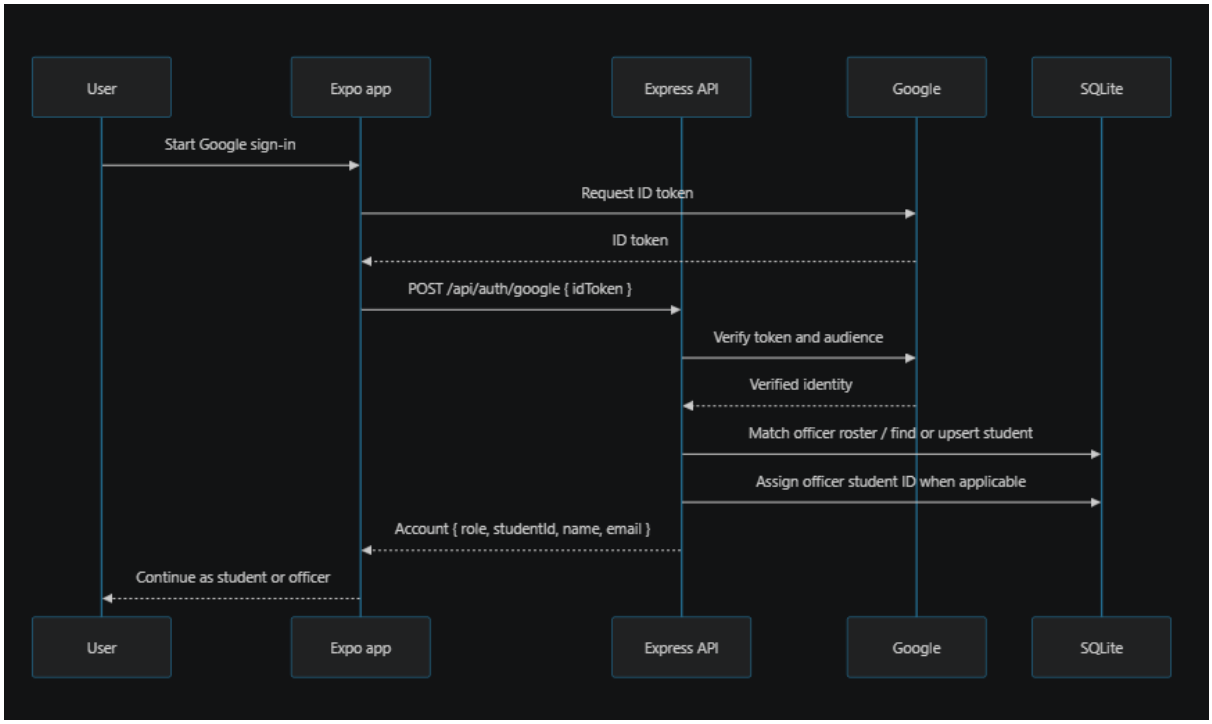
The EVENTRA app lets students and officers use the mobile or web interface to sign in, view events, and manage attendance. When the app needs data, it sends an HTTP request to the Express API. The API processes the request, checks the input, and reads or updates the SQLite database. It then sends a JSON response back to the app, which displays the result.

1. High-level flow

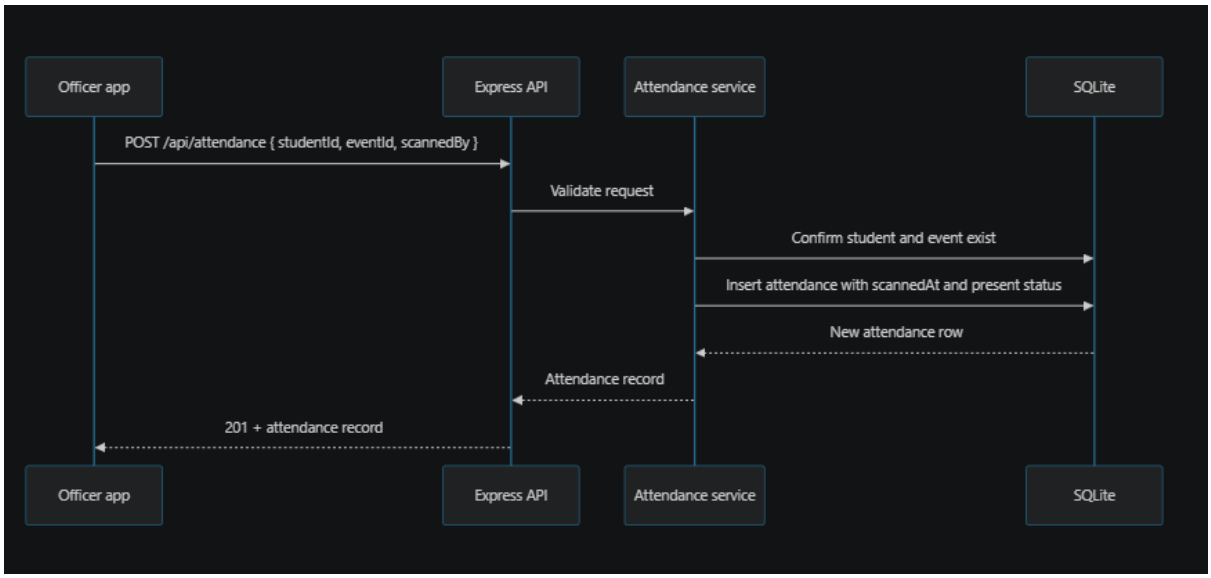




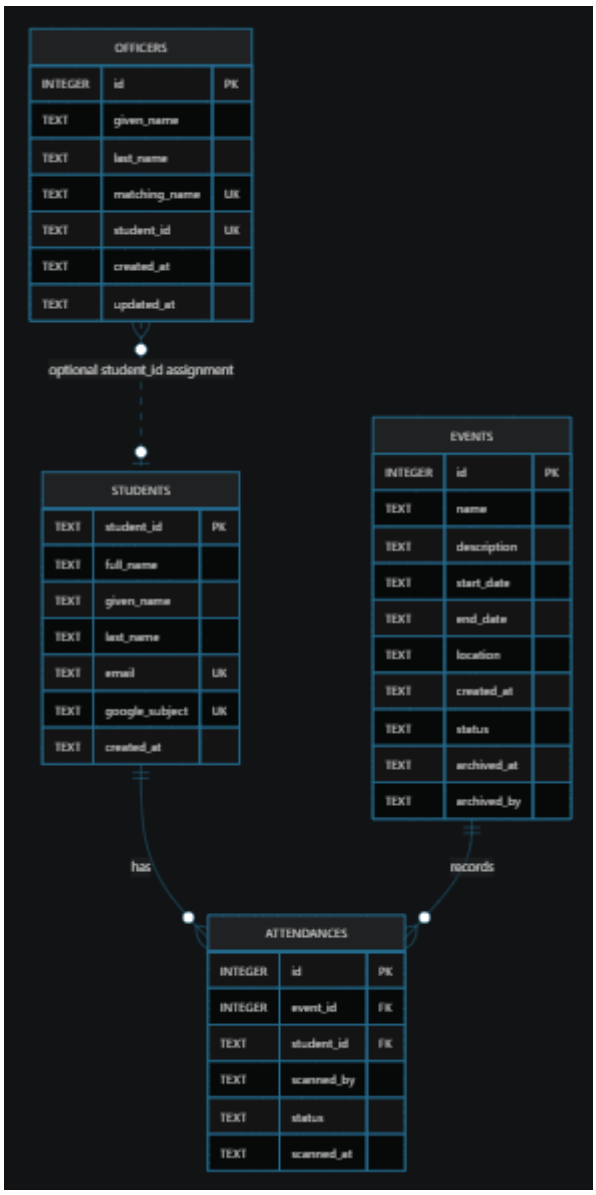
2. Main Workflows



3. Attendance Recording



4. Database Relationship diagram



TECH STACK USED

The EVENTRA School Event Attendance Management System was developed using a cross-platform mobile framework and a client-server architecture. The mobile application provides separate workflows for students and officers, while a REST API handles authentication, event management, and attendance records. The system stores its data in a local SQLite database.

Technology Stack	
Front End	
React Native	Builds the application's mobile user interface.
Expo	Provides the development and app runtime environment for React Native.
TypeScript	Support the app's component-based interface and typed application code.
Back End	
NodeJS	Runs the application's server.
ExpressJS	Provides the REST API for authentication, event, and attendance operations. Structures and type-checks the server code.
TypeScript	Structures and type-checks the server code.
Database	
SQLite	Stores student, officer, event, and attendance records.
NodeJS: sqlite	Connects the backend to SQLite using Node's built-in SQLite interface.
Libraries	
Expo Camera and react-native-qrcode-svg	Support QR code scanning and student attendance QR code generation.
Expo AuthSession	Starts Google sign-in from the app.
React Native Web	Enables the React Native interface to run on the web.

Google Auth Library	Verifies Google ID tokens on the server.
Expo DateTimePicker	Provides date selection for event forms.